

IMPA Tech

Machine Learning 2

Prof. Francisco Ganacim Prof. Daniel Yukimura

Gaussian Football n-D

Documentação Técnica

Gabriel Ferreira Silva

Letícia Aleixo Ferreira

Pedro Miguel Rocha Santos

Rio de Janeiro, 6 de junho de 2026

Índice

1	Visão Geral do Projeto	3
1.1	Construção dos Labels	3
1.2	Agregação por Clipe: <code>np.max</code>	3
2	Estrutura do Projeto	4
2.1	Árvore de Diretórios	5
3	Pipeline de Preparação de Dados	6
3.1	Dataset e Configuração	6
3.2	Baixando os Dados — <code>download_games.py</code>	6
3.3	Indexação dos Jogos — <code>build_index.py</code>	6
3.4	Geração de Clipes e Labels — <code>build_labels.py</code>	7
4	Pré-processamento	8
4.1	<code>video_slicer.py</code> — Fatiamento de Vídeo	8

1 Visão Geral do Projeto

O sistema automatiza a geração de *highlights* (a princípio, gols) de uma partida de futebol, utilizando tanto imagens quanto áudio (na forma de mel espectrogramas) extraídos de uma transmissão televisiva.

O problema é formulado sob a perspectiva de regressão. A cada clipe da partida associa-se um *arousal score*, um valor escalar contínuo que representa a intensidade emocional daquele trecho. Todo clipe cujo score supera um limiar pré-definido é considerado um highlight. Para isso, a partida é subdividida em n pares alinhados de clipe visual e mel espectrograma. O modelo recebe cada par como entrada e produz, como saída, um único valor escalar, o *arousal score* predito para aquele clipe.

1.1 Construção dos Labels

Cada partida possui anotações de momentos-chave fornecidas pelo dataset SoccerNet: gols, cartões, defesas, escanteios, entre outros. Para cada gol anotado no instante t_{gol} , associa-se uma curva gaussiana centrada nesse instante. O *arousal score* de qualquer instante t da partida é definido por:

$$a(t) = \exp\left(-\frac{(t - t_{\text{gol}})^2}{2\sigma^2}\right) \quad (1)$$

onde:

- t_{gol} é o instante do gol, onde o score atinge seu pico máximo $a(t_{\text{gol}}) = 1,0$;
- σ controla o raio de influência temporal do gol, determinado pelos parâmetros `-window_before` e `-window_after`;
- o score decai simetricamente à medida que t se afasta de t_{gol} , tendendo a zero fora da janela de highlight.

1.2 Agregação por Clipe: `np.max`

Ao fatiar a partida em clipes, cada clipe contém múltiplos frames, cada um com um score $a(t)$ calculado pela Equação 1. O score final associado ao clipe é o valor máximo entre todos os seus frames:

$$s_{\text{clipe}} = \max_{t \in \text{clipe}} a(t) \quad (2)$$

A escolha de `np.max` ao invés de `np.mean` é empírica. Considere um clipe de 5 segundos a 2 fps: esse clipe contém 10 frames. Se apenas 2 desses frames estão próximos do pico da gaussiana, a média dilui o sinal pelos 8 frames restantes, cujo score é próximo de zero. O máximo preserva o sinal mais forte do clipe, indicando que aquele clipe contém um momento de alta intensidade, independentemente de quantos frames não emocionantes coexistem nele.

2 Estrutura do Projeto

O projeto está organizado em cinco diretórios principais:

- `data/` — centraliza todos os dados do pipeline:
 - `raw/` — arquivos brutos baixados do SoccerNet;
 - `processed/` — clipes fatiados e mel espectrogramas, espelhando a estrutura de `raw/` no formato `liga/temporada/partida/tempo/`;
 - `labels/` — CSVs de índice e labels gerados pelos scripts de `src/`.
- `models/nn/` — arquiteturas de redes neurais implementadas;
- `preprocessing/` — utilitários de baixo nível para fatiamento de vídeo, extração de features e geração de mel espectrogramas;
- `src/` — scripts principais do pipeline: download, indexação e geração de labels;
- `outputs/` — reservado para resultados do modelo após o treinamento.

2.1 Árvore de Diretórios

```
gaussian_football/
├── data/
│   ├── labels/
│   │   ├── games_index.csv
│   │   └── labels_all.csv
│   ├── processed/
│   └── raw/
├── models/
│   └── nn/
│       ├── bidirectional_lstm.py
│       ├── cnnbilstm.py
│       └── vgg_16_lstm.py
├── outputs/
├── preprocessing/
│   ├── audio_mel_spectrogram.py
│   ├── datasets_mel_video.py
│   ├── image_resize.py
│   ├── image_to_array.py
│   ├── loaders.py
│   ├── mel_loader.py
│   ├── vgg16_feature_extract.py
│   ├── video_and_audio_get_sequences.py
│   ├── video_color.py
│   ├── video_get_sequence.py
│   ├── video_scorer.py
│   └── video_slicer.py
├── src/
│   ├── build_index.py
│   ├── build_labels.py
│   ├── download_games.py
│   └── treinamento.ipynb
├── README.md
└── requirements.txt
```

3 Pipeline de Preparação de Dados

3.1 Dataset e Configuração

Os dados utilizados neste projeto são provenientes do dataset SoccerNet, que requer autorização prévia para acesso aos arquivos de vídeo. Para fins acadêmicos e não comerciais, submetemos uma solicitação de uso à equipe do SoccerNet, que nos forneceu uma senha de acesso aos arquivos protegidos.

Foram utilizados jogos da Premier League inglesa (EPL) das temporadas 2014/2015, 2015/2016 e 2016/2017, totalizando 95 partidas distribuídas da seguinte forma:

- **Treino:** 58 jogos
- **Validação:** 19 jogos
- **Teste:** 18 jogos

Cada partida é composta por dois arquivos de vídeo (1_224p.mkv e 2_224p.mkv), correspondentes ao primeiro e segundo tempo, e um arquivo de anotações (Labels-v2.json) são provenientes do desafio de *action spotting* promovido pelo próprio SoccerNet, no qual anotadores identificaram eventos relevantes ao longo das partidas usando visão computacional.

A divisão dos jogos em treino, validação e teste segue a partição oficial do SoccerNet, definida nos arquivos SoccerNetGames.json, SoccerNetGamesTrain.json, SoccerNetGamesValid.json e SoccerNetGamesTest.json, disponíveis no repositório oficial do dataset. Esses arquivos especificam, para cada partida, o split ao qual ela pertence, garantindo que a divisão utilizada neste projeto seja reproduzível.

3.2 Baixando os Dados — download_games.py

O script download_games.py gerencia o download dos arquivos das partidas a serem utilizadas.

A função `normalize_splits` trata os diferentes protocolos de split do SoccerNet: a entrada pode ser um split explícito como "train", um atalho como "v1" (equivalente a ["train", "valid", "test"]) ou "all" (que inclui também o split "challenge"), ou ainda uma entrada inválida, que é rejeitada com uma mensagem de erro. A saída dessa função é sempre uma lista normalizada de splits válidos.

A função `game_already_downloaded` verifica se todos os arquivos esperados de uma partida já existem no disco. Ela é utilizada dentro de `download_games` para garantir que jogos já baixados sejam pulados em execuções subsequentes. Por fim, o modo `-dry_run` lista quais jogos seriam baixados e quais seriam pulados, sem efetivamente baixar nada, útil para inspecionar o estado do diretório antes de uma operação longa.

3.3 Indexação dos Jogos — build_index.py

O script build_index.py constrói o arquivo games_index.csv, que funciona como um organizador central do dataset. Ele é consultado por scripts subsequentes do pipeline para saber quais jogos estão disponíveis, onde seus arquivos estão no disco e a qual

split cada um pertence.

Para cada jogo, o CSV armazena as seguintes colunas:

- `game_id`: identificador legível e único do jogo, gerado pela função `make_game_id`;
- `league`: código da liga (ex: `epl`);
- `season`: temporada (ex: `2014-2015`);
- `split`: partição do dataset (`train`, `valid` ou `test`);
- `1_224p.mkv`, `2_224p.mkv`, `Labels-v2.json`: caminhos absolutos para cada arquivo da partida no disco;
- `valid`: booleano que indica se todos os arquivos esperados estão presentes no disco.

A função `make_game_id` gera o identificador a partir do caminho relativo do jogo no formato SoccerNet. A coluna `valid` é preenchida pela função `game_already_downloaded`, herdada do `download_games.py`, e indica se todos os arquivos esperados estão presentes no disco para aquela partida. Jogos com `valid=False` foram identificados mas ainda não foram baixados, são excluídos do pipeline de processamento pelos scripts seguintes.

3.4 Geração de Clipes e Labels — `build_labels.py`

O script `build_labels.py` é responsável tanto por gerar os clipes de vídeo e mel espectrograma (`generate_clips`) quanto por construir o arquivo `labels_all.csv` (`build_labels`), que para cada clipe armazena:

- `clip_path`: caminho para o arquivo de vídeo do clipe;
- `mel_path`: caminho para o mel espectrograma correspondente;
- `arousal_score`: score de intensidade do clipe, calculado via gaussiana;
- `game_id`, `season`, `split`: metadados da partida.

A função `generate_clips` itera sobre os dois tempos de cada partida e produz, para cada intervalo gerado pelo `VideoSlicer`, um clipe de vídeo e, quando o áudio não for silencioso, um mel espectrograma. Os arquivos são organizados na estrutura `half_1/video/` e `half_1/mel_spectrograma/` dentro do diretório processado de cada partida. É a função `build_labels` que realiza o cálculo do arousal score descrito na Seção 1, consultando a gaussiana gerada pelo `VideoScorerPreprocessor` e extraindo o valor máximo de cada intervalo via `np.max`.

O `main` verifica separadamente se os clipes já foram gerados e se os labels já existem no CSV. Isso garante que em caso de interrupção, o pipeline retome do ponto correto sem regenerar dados já processados nem perder progresso parcial.

4 Pré-processamento

4.1 `video_slicer.py` — Fatiamento de Vídeo

A classe `VideoSlicer` é responsável por fatiar vídeos em intervalos de tempo. Seu único parâmetro de inicialização é `n_slices`, o número de clipes a serem produzidos a partir do vídeo. O método `get_intervals` retorna uma lista de tuplas (`start`, `end`), cada uma representando os instantes inicial e final de um intervalo em segundos. O usuário deve fornecer ou a duração desejada de cada intervalo (`slice_length`), ou a quantidade desejada de intervalos (`n_slices`), nunca os dois simultaneamente. Ao montar os intervalos, o código garante que o `end` do último clipe não ultrapasse a duração total do vídeo via `end = min(start + length, total)`.

Referências

- [1] SoccerNet. *SoccerNet Data*. Disponível em: <https://www.soccer-net.org/data>. Acesso em: 6 de junho de 2026.
- [2] SoccerNet. *SoccerNet GitHub Repository*. Disponível em: <https://github.com/SoccerNet/SoccerNet/tree/main>. Acesso em: 6 de junho de 2026.